Scalar iteration uniqueness for iPRA

A complete arithmetic alternative and a small validated-checker refinement

Research report prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

The induction principle in the current scalar presentation of full first-order intuitionistic primitive recursive arithmetic can be replaced by unconditional uniqueness of a one-state iterator. From a derivation of $f(Sz) = h(f(z))$ under a fixed context, the new rule concludes $f(t) = I(t, f(0); u.h(u))$. It has no induction hypothesis and no separate base premise. We prove equivalence for all contextual first-order sequents, hence whole-domain bi-interpretability and equiconsistency with iPRA through the existing verified scalar bridge.

The main new proof closes the scalar equality-reflection gap explicitly left open in the incoming `scope_kernel` report. A modified finite-state ramp constructs predecessor using uniqueness. An explicit scalar accumulator then establishes $a + (b \dot{-} a) = b + (a \dot{-} b)$ and a faithful equality test, before pairing or general primitive recursion is used.

The size result is deliberately separate. A straightforward complete checker for the new uniqueness calculus is 3393 normalized Python bytes, versus 3322 for its matched induction control. Thus this new arithmetic basis is *not* a demonstrated improvement under the proof-checker-size criterion. A four-byte refinement of the existing, unchanged validated calculus reduces its complete checker from 870 to 866 bytes. Neither result is a proof of absolute minimality or a new overall compact-codec minimum. The mathematical equivalence is a written proof, not a new Lean or Rocq formalization.

Contents

1 Result, comparison, and provenance **3**

2 A complete native axiom and rule inventory **3**
2.1 The ordinary logical reading . . . 5

3 An induction-free scalar presentation **5**
3.1 The precise common background . . . 5

4 Zero-equation induction from uniqueness **6**
4.1 Retraction rigidity . . . 7
4.2 A numerical split suffices for induction . . . 7
4.3 The unconditional zero-equation case . . . 7

5 Scalar iteration uniqueness already defines predecessor **8**
5.1 Elementary identities by uniqueness . . . 8
5.2 Two finite clocks . . . 9
5.3 The modified ramp and its fusion law . . . 9
5.4 The scalar comparison recurrence . . . 10
5.5 Saturated iteration and predecessor . . . 10
5.6 The exact dependency boundary . . . 11

6 Actual equality from scalar iteration uniqueness **11**
6.1 A diagonal descent principle for numerical zero tests . . . 12
6.2 Two Boolean comparison identities . . . 12
6.3 An explicit scalar accumulator . . . 13
6.4 Commutativity of the arithmetic maximum expression . . . 13
6.5 Reflection and recovery of equation induction . . . 14

7 Equivalence, bi-interpretation, and the native calculus **15**
7.1 The two proof transformations . . . 15
7.2 What is inherited in the comparison with iPRA . . . 15
7.3 Replacing progress in the native presentation . . . 15
7.4 Scope of the result . . . 16

8 Implementation, scope reuse, and validation **16**
8.1 The uniqueness checker . . . 16
8.2 The separate four-byte refinement . . . 17
8.3 Arithmetic validation and assurance limits . . . 17

9 Further research questions **17**

10 Reproduction and files **18**

1 Result, comparison, and provenance

The task asks for a simpler complete axiomatization, with equiconsistency and bi-interpretability rather than just agreement about standard natural numbers. This report establishes a new complete arithmetic alternative and records precisely where its simplicity claim stops. Removing a local induction hypothesis simplifies the arithmetic inference interface. It does not, in the supplied implementation, make the complete checker smaller. Under the stricter checker-size objective, the mathematical part of the requested improvement remains unachieved.

The comparison is pinned to ProveIt commit

`11be5aabc6b07262046e2553419bc6b05271d178`.

The inspected project is `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. Its README and all six then-present incoming archives were inspected; the exact audit and source hashes accompany this report. The current repository already contains zero-computation normalization, six-rule presentations, one-term and template-free equality rules, and several introduction-mode refinements. These are not new contributions here [1, 5, 6].

The incoming `scope_kernel` report proves scalar uniqueness implies zero-equation induction by a retraction argument, but leaves predecessor and faithful numerical equality reflection as an open bootstrap problem [2]. We reproduce its retraction argument with attribution and complete that bootstrap. Recursion uniqueness as a general replacement for induction is classical. For example, Okada and Scott study uniqueness of iteration in a substantially richer typed setting [7]. The claim here is a completion relative to this repository's restricted scalar language, not historical priority for recursion uniqueness.

Complete checker	Bytes	Tokens	AST nodes	Gzip-9
Matched native equation-progress control	3322	1084	1141	984
New scalar uniqueness calculus	3393	1124	1175	1011
Existing full validated lexical codec	870	383	442	507
Same codec with scope reuse	866	381	439	504

All measurements use CPython 3.12.14, `ast.unparse`, and one final newline. Significant tokens omit comments and layout tokens; AST nodes count the complete parsed module. Gzip uses level 9 and timestamp zero, and is informational. The two pairs have different certificate formats. Only the first pair implements the new UR arithmetic. The 866-byte program retains the old calculus and is not a UR implementation. The repository's separately reported 516-byte profile has a different public syntax contract; no comparison across those interfaces is presented as a new minimum.

2 A complete native axiom and rule inventory

For a presentation with no ambient logical axioms, take one numerical sort and terms

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only u in h is bound. Formulas are

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A.$$

All substitutions avoid capture; contexts are finite reusable lists of formulas. The bound coordinates in D are distinct. The designated contradiction is $0 = S0$. No classical logical rule is implicit.

These eight patterns form the entire calculus. Its complete checker is `code/uniqueness/scalar_ur.py`.

Hypothesis. $\Gamma \vdash A$ when $A \in \Gamma$.

Two computation axioms. In every context,

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

Crossed transport in one term. For a numerical template $k(v)$,

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash k(s) = r}{\Gamma \vdash r = k(t)}. \tag{Tr}$$

Sources are unrestricted. In particular, literal zero is allowed.

Conditional-closure elimination.

$$\frac{\Gamma \vdash Q_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}. \tag{QE}$$

Paired elimination.

$$\frac{\Gamma \vdash D_{x,y} B(x, y) \qquad \Gamma, Q_y B(b, y) \vdash A}{\Gamma \vdash A}. \tag{DE}$$

The eigenparameter b is fresh for the retained context, conclusion, and fixed parameters of the matrix.

Paired introduction by cases only.

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y} B(x, y)}. \tag{DI}$$

Here z is fresh for Γ, t and all fixed matrix parameters. This rule treats atomic and compound matrices alike; there is no progress option and no literal-zero witness dispatch.

Scalar recursion uniqueness.

$$\frac{\Gamma \vdash f(Sz) = h[f(z)/u]}{\Gamma \vdash f(t) = I(t, f(0); u.h)}. \tag{UR}$$

The template $h(u)$ has no free z. The variable z is fresh for Γ, t and every passive parameter. It is a fresh numerical parameter, not a function variable. In particular, the rule's premise has no local arithmetic assumption.

There is no further equality axiom, induction axiom, successor axiom, structural proof instruction, or logical axiom. Reflexivity follows by using $I(0, t; u.0) = t$ as both transport premises with $k(v) = v$. Symmetry follows by transporting $s = t$ along $s = s$ with that same template. Transitivity and term congruence then follow directly. Full formula transport is recovered by structural induction on the finite formula syntax, using DI, DE, and QE. Weakening, renaming, substitution, and assumption cut are admissible by induction on derivations. UR is stable under parameter substitution after its eigenparameter is freshened.

2.1 The ordinary logical reading

The readings are

$$Q_y B := B(0) \to \forall y\, B(y), \qquad D_{x,y} B := \exists x (B(x,0) \to \forall y\, B(x,y)).$$

Ordinary intuitionistic logic is reconstructed without any use of equation progress. For clarity, a compact description of its macros is given here. The exact inherited derivations are included in `reference/inherited_choice_basis.tex`, especially the sections on cases, equality, binder congruence, and the entire ordinary logical interface [3].

First replace a Q formula, when needed inside a macro, by an equivalent D formula with an unused first coordinate. Set $C(v,a,b) = I(v,a;u.b)$. On equations define a gate $G_v(s=t)$ as $C(v,0,s) = C(v,0,t)$ and extend it through D. It is provable at zero and equivalent to the original formula at a successor. A formula selector $M_v(A,B)$ is built by selecting the two equation endpoints with C, extending through aligned D binders, and padding the shallower input by $D_{w,y}G_y$. Thus its zero instance is equivalent to A, and every successor instance to B. Put $p(z) = I(z,0;u.C(u,1,0))$, so $p(1) = 1$ and $p(2) = 0$ by computation. With all displayed extra bound variables fresh, the macros are

$$\begin{aligned}
\bot_* &:= (0 = S0),\\
A \to_* B &:= D_{w,z} M_z(A,B),\\
A \wedge_* B &:= D_{w,z} G_z M_{p(z)}(A,B),\\
\exists_* x\, A &:= D_{x,y} G_y A,\\
A \vee_* B &:= \exists_* v\, M_v(A,B),\\
\forall_* x\, A(x) &:= A(0) \wedge_* D_{w,z} G_z A(z).
\end{aligned}$$

The rules for these macros are precisely intuitionistic natural deduction: assumption; introduction and elimination for $\wedge, \vee, \to, \forall, \exists$; and explosion from $\bot$. Quantifier introduction and existential elimination have their usual fresh-parameter conditions. For $\vee$-elimination both branches must prove the same conclusion. No rule of excluded middle or double-negation elimination is present. The numerical case rule is derived using DI with an unused witness, then DE and QE. It gives coverage. Under $0 = S0$, selector transport gives every equation, and external formula-structural induction gives explosion. These observations justify using ordinary intuitionistic reasoning in the arithmetic proof that follows without relying on the induction rule being replaced.

3 An induction-free scalar presentation

The new arithmetic principle is unconditional uniqueness of scalar iteration. The general idea of replacing induction by recursion uniqueness is classical. The result proved here is more specific: uniqueness for a *single scalar state with no counter argument in the update* already suffices, in the original scalar language. It closes the explicit equality-reflection gap in the incoming `scope_kernel` report. It does not rely on a general primitive recursion compiler or on numeric pairing before induction has been recovered.

3.1 The precise common background

Use one numerical sort and fixed finite term syntax

$$t ::= x \mid 0 \mid S(t) \mid I(n,a;u.h).$$

The binder binds u only in h. Equivalently, each finite earlier-stage body $h(u, \vec{p})$ indexes one first-order function symbol $I_h(n, a, \vec{p})$. Bodies are not function variables, and the theory postulates no universal evaluator for primitive-recursive descriptions. All substitutions avoid capture.

The surrounding logic is intuitionistic first-order logic with actual equality and genuine falsity $\bot$. Its usual logical constructors are $\wedge, \vee, \to, \forall, \exists, \bot$; negation abbreviates implication to $\bot$. In particular, no double-negation elimination or excluded middle for arbitrary formulas is assumed.

The nonlogical axioms are the universally closed instances

$$\begin{aligned} I(0, a; u.h) &= a, \\ I(Sn, a; u.h) &= h[I(n, a; u.h)/u], \\ \forall n\,(n = 0 \vee \exists m\, n = Sm), & \\ 0 = S0 &\longrightarrow \bot. \end{aligned}$$

No successor injectivity axiom, equality characteristic, primitive pairing, or induction axiom occurs in this background.

The uniqueness theory T_{UR} adds the single inference schema

$$\boxed{\frac{\Gamma \vdash f(Sz) = h(f(z))}{\Gamma \vdash f(t) = I(t, f(0); u.h(u))}.} \tag{UR}$$

Here $f(z)$ and $h(u)$ are fixed finite term templates. The update has no free z. The variable z is fresh for the retained context, the target t, and all passive parameters. Crucially, the premise is proved in Γ itself: there is no additional induction hypothesis. The seed $f(0)$ is read from the same template, so there is no separate base premise. This is an inference schema over finite syntax, not quantification over a sort of functions.

Let T_{eq} use the same background and replace UR by contextual induction on each literal equation $s(n) = r(n)$:

$$\frac{\Gamma \vdash s(0) = r(0) \qquad \Gamma, s(z) = r(z) \vdash s(Sz) = r(Sz)}{\Gamma \vdash s(t) = r(t)}, \tag{EI}$$

with the same usual freshness condition.

Theorem 3.1 (Scalar uniqueness completion)**.** *For every first-order sequent over the scalar language,*

$$\Gamma \vdash_{T_{\mathrm{UR}}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{T_{\mathrm{eq}}} A.$$

Consequently T_{UR} is bi-interpretable and equiconsistent with full first-order iPRA. The arithmetic comparison uses the entire numerical domain and actual equality; both round trips have identity graph $x = y$.

The hard implication is proved below. The order of the proof prevents the usual circularity: UR first gives zero-equation induction and a predecessor, then a scalar accumulator gives equality reflection, and only then is arbitrary equation induction recovered. General primitive recursion is compiled after that point.

4 Zero-equation induction from uniqueness

The argument in this section is the retraction argument already present in the incoming `scope_kernel` report. It is reproduced to make the dependency on equality reflection exact. It uses neither predecessor nor a general numerical equality characteristic.

All calculations are under one fixed retained first-order context Γ. Fresh displayed variables are absent from Γ and all passive parameters. UR first gives

$$I(n, 0; u.Su) = n. \tag{Id}$$

Use the template $f(z) = z$ and update $h(u) = Su$; its recurrence premise is reflexivity. Common-update uniqueness, obtained by applying UR separately to two terms and replacing equal seeds, is used below.

4.1 Retraction rigidity

Let $e(x)$ be a fixed scalar term template. Suppose Γ proves

$$e(0) = 0, \qquad e(e(x)) = e(x), \qquad e(S(e(x))) = S(e(x)). \tag{Ret}$$

Then Γ proves $e(t) = t$ for every target t.

To see this, put $h(u) = e(S(e(u)))$ and $r(n) = I(n, 0; u.h(u))$. Idempotence in Ret gives $e(h(u)) = h(u)$ and $h(e(u)) = h(u)$. Therefore the template $f(n) = e(r(n))$ has initial value zero and unconditional recurrence

$$f(Sn) = e(h(r(n))) = h(r(n)) = h(e(r(n))) = h(f(n)).$$

UR gives $e(r(n)) = I(n, 0; u.h(u)) = r(n)$. The last equation of Ret now yields

$$r(Sn) = h(r(n)) = S(e(r(n))) = S(r(n)), \qquad r(0) = 0.$$

A second application of UR, followed by Id, gives $r(n) = n$. Replacing $r(n)$ by n in the previous equality proves $e(n) = n$. No induction was used, and no claim that the standard finite successor orbit exhausts a model was used.

4.2 A numerical split suffices for induction

For a fixed formula $A(n)$, suppose

$$\Gamma \vdash A(0), \qquad \Gamma, A(z) \vdash A(Sz).$$

Suppose moreover that $A(n)$ is decidable under Γ and that a term $e(n)$ satisfies $e(n) = n$ under $A(n)$ and $e(n) = 0$ under $\neg A(n)$. Cases give $A(e(n))$: in the first case use $A(n)$, and in the second use the proved base $A(0)$. Applying the true-case identity at $e(n)$ gives $e(e(n)) = e(n)$. The induction-step derivation, instantiated at $e(n)$, gives $A(S(e(n)))$, hence $e(S(e(n))) = S(e(n))$. The base similarly gives $e(0) = 0$. Retraction rigidity proves $e(n) = n$; transporting $A(e(n))$ gives $A(n)$.

This is a metatheorem about a justified numerical split. It supplies no decision procedure, comprehension principle, or induction schema for arbitrary quantified formulas.

4.3 The unconditional zero-equation case

Write $1 = S0$ and $C(v, a, b) = I(v, a; u.b)$. The two computations give $C(0, a, b) = a$ and $C(Sv, a, b) = b$. The separation axiom $0 = 1 \to \bot$ implies $Sv \neq 0$: otherwise applying $w \mapsto C(w, 0, 1)$ gives $1 = 0$, hence a contradiction. Coverage therefore decides $q(n) = 0$ for every fixed scalar term $q(n)$. The term

$$e(n) = C(q(n), n, 0)$$

has exactly the split required in the preceding subsection. Consequently the following contextual rule is derived:

$$\frac{\Gamma \vdash q(0) = 0 \qquad \Gamma, q(z) = 0 \vdash q(Sz) = 0}{\Gamma \vdash q(t) = 0}. \tag{ZI}$$

Every substitution and case distinction is intuitionistic. The retained context can contain arbitrary quantified formulas, provided the fresh induction variable is absent from it.

5 Scalar iteration uniqueness already defines predecessor

This section gives the predecessor portion of the completion of the scalar-uniqueness direction identified in the incoming `scope_kernel` report. Its new ingredient is a modified scalar ramp whose invariant is a direct fusion law. Equality reflection is proved independently in Section 6.

Work in ordinary intuitionistic first-order logic with actual equality. Terms are generated by variables, 0, S, and $I(n, a; u.h)$, with only u bound in h. Assume the two iterator equations,

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u],$$

and numerical coverage $n = 0 \vee \exists m\, n = Sm$. Replace equation induction by the contextual scalar uniqueness rule

$$\frac{\Gamma \vdash f(Sz) = h[f(z)/u]}{\Gamma \vdash f(t) = I(t, f(0); u.h)} \quad \text{(UR)},$$

where z is fresh for Γ and the passive parameters, $f(z)$ is a term template, and $h(u)$ has no free z. All bodies are finite earlier syntax. A useful derived rule says that f and g are equal whenever they have provably equal initial values and the same unconditional autonomous update. Apply UR to both and transport the initial value. No induction hypothesis is permitted in the recurrence premise.

Theorem 5.1. *The preceding theory defines a finite scalar term P and proves $P(0) = 0$ and $P(Sn) = n$. Consequently it proves successor injectivity. The derivation does not use separation of 0 and $S0$.*

5.1 Elementary identities by uniqueness

Write $1 = S0$, set $C(n, a, b) = I(n, a; u.b)$, $Z(n) = C(n, 1, 0)$, and $a + b = I(b, a; u.Su)$. Coverage gives $Z(n) = 0 \vee Z(n) = 1$ and

$$C(0, a, b) = a, \quad C(Sn, a, b) = b, \quad C(e, Sa, Sb) = S(C(e, a, b)).$$

The last identity follows directly by cases on e. All commutative-monoid laws for addition follow from common-update uniqueness. In detail, n has initial value 0 and update S, so $0 + n = n$; $S(a+n)$ has initial value Sa and update S, so $Sa+n = S(a+n)$. Thus $n+a$ has initial value a and update S, proving $n + a = a + n$. Finally $a + (b + n)$ has initial value $a + b$ and update S, proving associativity. These facts justify the usual finite sum notation $2a = a + a$ and $3a = a + a + a$.

For any fixed body h, the following are also common-update uniqueness proofs:

$$I(m + n, a; h) = I(n, I(m, a; h); h), \qquad I(Sn, a; h) = I(n, h(a); h).$$

For the first compare the two sides as functions of n, with update h and initial value $I(m, a; h)$; for the second the initial value is $h(a)$. A constant function fixed by h is likewise the corresponding iterator.

5.2 Two finite clocks

Define

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

Coverage on the counter of N proves that $N(n)$ is 0 or 1. Hence $Z(Z(N(n))) = N(n)$ and $N(n) + Z(N(n)) = 1$ by two finite cases. The computation rule gives $N(n+2) = N(n)$. The function $N(2n)$ has initial value 0 and identity update, whereas $N(3n)$ has initial value 0 and update Z. UR therefore gives

$$N(2n) = 0, \qquad N(2n+1) = 1, \qquad N(3n) = N(n).$$

The range of j lies in $\{0, 1, 2\}$: at $u = 0$ its value is 1; at $u = Sv$ the term $Z(u)$ is zero and $2N(u)$ is 0 or 2. Coverage on the counter of M therefore proves that its range also lies in $\{0, 1, 2\}$. Direct computation gives

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

Thus $M(n+3) = M(n)$ by three computations and the three cases for $M(n)$. UR with the identity update gives $M(3n) = 0$, and one further computation gives $M(3n+1) = 1$.

5.3 The modified ramp and its fusion law

Use the update

$$g(u) = u + N(M(u)) + 1, \qquad G(n) = I(n, 0; u.g(u)).$$

It differs from the older ramp update $u + M(u) + 1$ away from its intended orbit. That difference makes the following law valid at every argument:

$$M(g(u)) = Z(M(u)). \tag{1}$$

Indeed, when $M(u) = 0$, $g(u) = u + 1$ and the new M value is 1; when $M(u) = 1$, $g(u) = u + 2$ and the new value is 0; when $M(u) = 2$, $g(u) = u + 1$ and the new value is again 0. Consequently $M(G(n))$ has initial value 0 and update Z. Uniqueness, without any induction hypothesis, proves

$$M(G(n)) = N(n). \tag{2}$$

As N fixes the two Boolean values, this gives

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3, \qquad G(n+3) = G(n) + N(n) + 4. \tag{3}$$

The last two equations follow by two or three computations and the two cases for $N(n)$.

The functions $G(2n)$ and $3n$ both start at zero and have update $v \mapsto v + 3$, so UR yields $G(2n) = 3n$. For the less immediate scaling equation, put

$$f(n) = G(3n), \qquad k(n) = 3G(n) + N(n), \qquad H_0(v) = v + M(v) + 4.$$

Equation (2) and parity scaling give $M(f(n)) = N(n)$. The identities $M(3a) = 0$ and $M(3a+1) = 1$, together with the two cases for $N(n)$, give $M(k(n)) = N(n)$. Equations (3) show $f(Sn) = H_0(f(n))$. In those same two Boolean cases,

$$\begin{aligned} k(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)) \\ &= 3G(n) + 2N(n) + 4 = H_0(k(n)). \end{aligned}$$

Both start at zero. A second common-update uniqueness proof therefore gives

$$G(2n) = 3n, \qquad G(3n) = 3G(n) + N(n). \tag{4}$$

5.4 The scalar comparison recurrence

Set $H(u) = C(N(u), G(u), u)$. From (4) and the parity identities,

$$H(2u) = 3u, \quad H(1) = 1, \quad H(3u) = 3H(u).$$

For the last identity split $N(u) = 0$ or 1; in the zero case use $G(3u) = 3G(u)$, and in the one case both sides retain the corresponding odd argument. No division theorem is used. The functions $I(y, 3a; H)$ and $3I(y, a; H)$ have the same initial value $3a$ and update H, hence

$$I(y, 3a; H) = 3I(y, a; H), \qquad I(y, 1; H) = 1.$$

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N(I(y, E(x); H)).$$

The rotation identity and the displayed equations yield

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{5}$$

For the final equality, rotate the first iterator step at the initializer $2E(x)$, replace $H(2E(x))$ by $3E(x)$, move the factor three through the iterator, and use $N(3v) = N(v)$.

5.5 Saturated iteration and predecessor

Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

Equation (5) and selector compatibility with successor give

$$B_{Sm}(Su) = SB_m(u), \qquad B_{Sm}(0) = 1.$$

As functions of t, both $W(St, Sm)$ and $SW(t, m)$ start at one and have update B_{Sm}. Therefore UR proves

$$W(St, Sm) = SW(t, m). \tag{6}$$

The function $n \mapsto L(Sn, n)$ starts at zero and has identity update by (5), so it is identically zero by UR. By the already universal equation (6), $n \mapsto W(n, n)$ starts at zero and has update S, so it equals n. It follows that

$$W(Sn, n) = B_n(W(n, n)) = C(L(Sn, n), n, Sn) = n. \tag{7}$$

Finally put

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big).$$

The zero equation is immediate. For fixed m, the function

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big)$$

has the same initial value and update as $W(t, m)$, because (5) identifies its update pointwise with B_m. UR yields $V(t, m) = W(t, m)$. Using (7),

$$P(Sm) = V(Sm, m) = W(Sm, m) = m.$$

Applying P to $Sx = Sy$ now gives $x = y$.

5.6 The exact dependency boundary

Every appeal to induction in the predecessor bootstrap has been replaced by a specified unconditional common-update uniqueness instance. All case distinctions use coverage and finite ranges; no separation axiom, equality characteristic, or induction on an equation has been smuggled in. One also obtains $d(x,y) = I(y,x;u.Pu)$ and

$$d(Sx,Sy) = d(x,y), \qquad d(a+n,n) = a,$$

again by common updates P and the identity, respectively. Consequently addition is cancellative, and $m(x,y,z) = d(x+z,y)$ is a Mal'cev operation: $m(x,x,z) = z$ and $m(x,y,y) = x$.

The equality-reflection step needs a different argument. The older proof used

$$C\big(\mathrm{sg}(d(x,y)), x, y + d(x,y)\big) = x, \qquad \mathrm{sg}(v) = C(v,0,1).$$

Its familiar proof invokes the equality at y as an induction hypothesis. The left side has a counter-dependent reset recurrence, not an immediate autonomous scalar recurrence. It is not imported into this bootstrap. Instead, Section 6 constructs an explicit scalar accumulator using Boolean diagonal descent, proves a maximum commutativity identity, and obtains actual equality reflection from it.

6 Actual equality from scalar iteration uniqueness

Work in intuitionistic first-order logic with actual equality, the scalar iterator computation equations, numerical zero/successor coverage, and $0 \neq S0$. Assume the contextual scalar uniqueness rule

$$\frac{\Gamma \vdash f(Sz) = h(f(z))}{\Gamma \vdash f(t) = I(t, f(0); u.h(u))}, \tag{UR}$$

where z is fresh for Γ, t and all passive parameters. The update h has no active counter argument. Every displayed function in what follows is a fixed finite scalar term abbreviation; there are no function variables and no hidden product-state iteration.

The preceding scalar predecessor construction supplies a term P with $P0 = 0$ and $P(Sx) = x$. Addition, defined by $a + b = I(b,a;u.Su)$, satisfies the commutative-monoid equations by common-update uniqueness. Define

$$d(a,b) = I(b,a;u.Pu), \qquad Z(a) = C(a,1,0), \qquad \mathrm{sg}(a) = C(a,0,1), \qquad C(v,a,b) = I(v,a;u.b).$$

The following elementary equations use only UR and computation:

$$\begin{aligned} d(a,0) &= a, & d(a,Sb) &= P(d(a,b)), \\ d(Sa,Sb) &= d(a,b), & d(a,a) &= 0, & d(0,a) &= 0. \end{aligned} \tag{1}$$

For cancellation, the two terms $b \mapsto d(Sa,Sb)$ and $b \mapsto d(a,b)$ have common initial value a and common update P. The diagonal $a \mapsto d(a,a)$ has the identity update and initial value zero. The constant-zero function and $a \mapsto d(0,a)$ have common update P and initial value zero. Thus no equation-induction assumption is hidden here.

6.1 A diagonal descent principle for numerical zero tests

UR implies the following constant-recurrence principle:

$$\Gamma \vdash f(Sz) = f(z) \quad \Longrightarrow \quad \Gamma \vdash f(t) = f(0). \tag{Const}$$

Apply UR with identity update to f and to the constant function $f(0)$.

Fix a finite arity $r \geq 1$ and a scalar term $q(x_1, \ldots, x_r)$. Suppose that, with any passive parameters fixed,

$$q(Sx_1, \ldots, Sx_r) = q(x_1, \ldots, x_r)$$

and that q is zero whenever any of its displayed arguments is zero. Then

$$q(x_1, \ldots, x_r) = 0. \tag{DD}$$

Indeed, coverage in each argument gives $q(Px_1, \ldots, Px_r) = q(x_1, \ldots, x_r)$: if any argument is zero, both sides are zero; otherwise the successor identity applies. Now put

$$f(k) = q(d(x_1, k), \ldots, d(x_r, k)).$$

The defining equation for d gives $f(Sk) = f(k)$, so Const makes f constant. At $k = x_1$ its first argument is $d(x_1, x_1) = 0$; hence $f(x_1) = 0$. At $k = 0$ it is the desired $q(x_1, \ldots, x_r)$. This argument uses scalar common-update uniqueness. It does not quantify over tuples or invoke a simultaneous-recursion rule.

For Boolean-valued terms p, q, use

$$\mathrm{or}(p, q) = C(p, q, 1), \qquad \mathrm{xor}(p, q) = C(p, q, Z(q)).$$

Here Boolean-valued means that the term is provably either 0 or 1. Finite cases show that $\mathrm{xor}(p, q) = 0$ if and only if $p = q$. This finite Boolean fact is not reflection of arbitrary numerical equality.

6.2 Two Boolean comparison identities

First, DD gives

$$Z(d(Sn, a)) = \mathrm{sg}(d(a, n)). \tag{B1}$$

To verify its DD premises, let $q(n, a)$ be the Boolean xor of the two displayed sides. Simultaneously replacing n, a by Sn, Sa cancels one successor in each subtraction, so $q(Sn, Sa) = q(n, a)$. If $a = 0$, both sides are zero. If $n = 0$, split a into zero and successor: the former case again gives two zeros, while $a = Sb$ gives $d(1, Sb) = d(0, b) = 0$ and $\mathrm{sg}(Sb) = 1$, so both sides are one. Thus both zero-coordinate boundaries vanish and DD applies.

Second, DD gives

$$Z\big(d(d(x, a), d(b, a))\big) = \mathrm{or}\big(Z(d(x, a)), Z(d(x, b))\big). \tag{B2}$$

Again use the Boolean xor $q(x, a, b)$ of the two sides. When all three arguments are successors, simultaneous cancellation reduces the expression to $q(x, a, b)$. The three zero-coordinate boundaries are as follows. If $x = 0$, both displayed sides are one. If $a = 0$, the left side is $Z(d(x, b))$, and the right side is $\mathrm{or}(Z(x), Z(d(x, b)))$; coverage on x makes them equal. If $b = 0$, the left side is $Z(d(x, a))$, and the right side is $\mathrm{or}(Z(d(x, a)), Z(x))$; coverage on x again proves equality. In both latter arguments, the $x = 0$ branch makes the other Boolean one, whereas the successor branch

makes $Z(x) = 0$. Consequently DD proves the vanishing of the xor, and Boolean cases give the stated identity.

Coverage and the successor equation for addition also give

$$\mathrm{or}(\mathrm{sg}(u), \mathrm{sg}(v)) = \mathrm{sg}(u + v). \tag{B3}$$

For $u = 0$ this reduces to the sign of v; for successor u, both sides are one.

6.3 An explicit scalar accumulator

Define the two scalar terms

$$\begin{aligned} \mu(n, a) &= d(n, d(n, a)), \\ \phi(n, a, b) &= \mu(n, a) + \mu(d(n, a), d(b, a)). \end{aligned} \tag{2}$$

These are finite compositions of d and addition. In particular, ϕ is not introduced by a counter-dependent recursion. We now prove its useful counter-dependent equation from its explicit term.

Coverage on a and the monus equations give

$$P(d(Sn, a)) = d(n, a). \tag{Pd}$$

At $a = 0$ this is $P(Sn) = n$. At $a = Sa'$ it follows from $d(Sn, Sa') = d(n, a')$ and $P(d(n, a')) = d(n, Sa')$. Split the value $r' = d(Sn, a)$ into zero and successor. If $r' = 0$, then $d(n, a) = 0$ by Pd, so $\mu(Sn, a) = Sn$ and $\mu(n, a) = n$. If $r' = Sr$, then $d(n, a) = r$ and $\mu(Sn, a) = d(Sn, Sr) = d(n, r) = \mu(n, a)$. In both cases,

$$\mu(Sn, a) = \mu(n, a) + Z(d(Sn, a)). \tag{M}$$

Put $h = d(b, a)$ and repeat the same split of $r' = d(Sn, a)$. If $r' = 0$, the first argument of the second μ term is zero at both n and Sn, and $\phi(Sn, a, b) = Sn$, $\phi(n, a, b) = n$. If $r' = Sr$, the first summand is unchanged, while M gives

$$\mu(Sr, h) = \mu(r, h) + Z(d(Sr, h)).$$

Associativity therefore proves, in both cases,

$$\phi(Sn, a, b) = \phi(n, a, b) + Z(d(d(Sn, a), d(b, a))).$$

Apply B2 at $x = Sn$, then B1 to each Boolean on its right, and finally B3. The resulting recursion specification is

$$\boxed{\phi(0, a, b) = 0, \qquad \phi(Sn, a, b) = \phi(n, a, b) + \mathrm{sg}(d(a, n) + d(b, n)).} \tag{Acc}$$

All identities leading to Acc used scalar uniqueness, coverage, and finite case analysis. No equality-reflection or induction theorem has yet been used.

6.4 Commutativity of the arithmetic maximum expression

Put

$$m(a, b) = a + d(b, a).$$

The name is descriptive only; its order-theoretic properties are not assumptions. Four zero/successor cases prove

$$m(a, b) = m(Pa, Pb) + \mathrm{sg}(a + b). \tag{Red}$$

At $(0,0)$ both sides are zero. At $(0, Sb)$ and $(Sa, 0)$ it is the successor equation for addition. At (Sa, Sb), simultaneous subtraction cancellation gives $m(Sa, Sb) = Sa + d(b,a) = S(m(a,b))$, and the sign of the sum is one.

For fixed a, b, form the scalar terms

$$F(n) = m(d(a,n), d(b,n)) + \phi(n,a,b),$$
$$G(n) = m(d(b,n), d(a,n)) + \phi(n,a,b).$$

Red, Acc, and commutativity of addition give $F(Sn) = F(n)$ and $G(Sn) = G(n)$. Hence Const gives

$$F(n) = m(a,b), \qquad G(n) = m(b,a).$$

At $n = b$, the equality $d(b,b) = 0$ and the identities $m(c,0) = c = m(0,c)$ make both terms equal, by these equations, to $d(a,b) + \phi(b,a,b)$. It follows that

$$\boxed{a + d(b,a) = b + d(a,b).} \tag{Max}$$

This is the common-accumulator argument for maximum commutativity. The scalar-specific ingredient is the explicit accumulator in (2) and its derivation through Boolean DD; no prior availability of arbitrary primitive recursion is required.

6.5 Reflection and recovery of equation induction

Define

$$\Delta(a,b) = \mathrm{sg}(d(a,b) + d(b,a)).$$

Coverage and separation imply $\mathrm{sg}(u) = 0 \leftrightarrow u = 0$. They also imply $u + v = 0 \leftrightarrow (u = 0 \wedge v = 0)$: a successor in either summand would make the sum a successor, contradicting separation. Thus $\Delta(a,b) = 0$ forces $d(a,b) = 0 = d(b,a)$. Max then yields

$$a = a + d(b,a) = b + d(a,b) = b.$$

Conversely $a = b$ gives both subtractions zero. Therefore

$$\boxed{\Delta(a,b) = 0 \iff a = b, \qquad \Delta(a,b) = 0 \ \vee \ \Delta(a,b) = 1.} \tag{EqRef}$$

The equality on the right of EqRef is the logic's actual equality on the whole numerical domain. Nothing restricts a, b to standard numerals or to the image of an encoding.

Finally, the independently proved UR retraction lemma gives contextual induction for all zero equations $q(n) = 0$. Given an induction premise for any equation $s(n) = t(n)$, use EqRef to transfer its base and step to the zero equation $\Delta(s(n), t(n)) = 0$, apply that zero-induction rule, and transfer the conclusion back. This recovers contextual literal-equation induction in arbitrary retained first-order contexts. The reverse derivation of UR from literal-equation induction is immediate by induction on $f(n) = I(n, f(0); h)$. Consequently scalar UR and contextual equation induction are equivalent over the common scalar computation, coverage, and separation background, once the preceding UR predecessor bootstrap is included.

7 Equivalence, bi-interpretation, and the native calculus

7.1 The two proof transformations

The preceding sections explicitly derive EI inside T_{UR}. For any requested instance $s(n) = r(n)$, replace it by $\Delta(s(n), r(n)) = 0$ using EqRef, apply ZI to the transferred base and step, and use EqRef again at the target. The context Γ has remained fixed throughout. Thus this argument applies to contextual induction, not only to closed unconditional theorems.

Conversely, in T_{eq} assume the premise of UR and apply EI to

$$f(n) = I(n, f(0); u.h(u)).$$

The base is the zero computation and symmetry. At a successor, the retained recurrence premise, congruence applied to the induction equation, and successor computation give

$$f(Sn) = h(f(n)) = h(I(n, f(0); h)) = I(Sn, f(0); h).$$

This derives UR. Replacing each use of one rule by its displayed finite derivation proves equality of all derivable sequents. The transformation requires only finite manipulation of fixed terms and proofs.

7.2 What is inherited in the comparison with iPRA

The supplied project already proves that this ordinary scalar T_{eq} is bi-interpretable with full first-order iPRA. The pinned reference files are `inherited_ordinary_bridge.tex`, `equality_reflection.tex`, and `primitive_recursion.tex` under `Research/certified_zero/reference/`, at repository commit `11be5aabc6b07262046e2553419bc6b05271d178`. The exact point at which that bridge is used here is after EI has been derived independently from UR.

More explicitly, each fixed scalar iterator has its usual primitive-recursive interpretation in iPRA. Equation induction proves its uniqueness, hence validates UR. In the other direction, EqRef and the selector assign a Boolean term to every fixed quantifier-free formula. Truth tables turn its induction into equation induction. The inherited arithmetic construction supplies scalar pairing and projection terms with their inverse equations. An ordinary primitive recursion can then be simulated by iterating the paired state consisting of its counter and current value. This last use of pairing is legitimate because equation induction has already been recovered.

All quantifiers in these translations range over the whole numerical domain. Equality translates as actual equality. The two translations replace only fixed primitive-recursive operation descriptions by the proved corresponding terms, and the defining recurrences identify the round trips. Their numerical isomorphism is $x = y$. Since the identity translation between T_{UR} and T_{eq} preserves every sequent, composing it with the pinned scalar bridge preserves these same whole- domain and identity-composite properties. Mutual translation of a derivation of $\bot$ gives equiconsistency.

7.3 Replacing progress in the native presentation

There is also a direct native corollary. Retain the original terms and native formula constructors

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A,$$

and their ordinary readings

$$Q_y B = B(0) \to \forall y\, B(y), \qquad D_{x,y} B = \exists x\, (B(x, 0) \to \forall y\, B(x, y)).$$

Keep Hyp, the two computation equations, equality transport, QE, DE, and DI by cases; replace equation-progress introduction by UR. This gives exactly the native sequents of the original calculus.

The logical reconstruction needed for this statement is independent of the deleted progress rule. The pinned `inherited_choice_basis.tex` derives numerical cases, binder congruence, the ordinary intuitionistic logical interface, and explosion using DI by cases, QE, DE, computation, and transport. With crossed one-term transport, the ordinary equality rules and equation-template transport are first recovered; transport through Q and D follows by structural induction on formulas using the case rules. These are inductions on finite syntax in the metatheory, not hidden numerical induction instances.

Thus the ordinary UR derivation above can be translated into native macros without an appeal to progress. It recovers EI and therefore the old progress rule. Conversely, old equation progress gives EI and the displayed proof of UR. This establishes the native equivalence. It does not by itself imply that a particular Python encoding is smaller; that claim requires the complete checker and its matched measurement.

7.4 Scope of the result

The theorem is a written internal derivation, not a proof-assistant formalization. Numerical checks of the displayed terms are useful implementation checks but do not verify the theorem in nonstandard or intuitionistic models. No claim of absolute minimality is made. The mathematical advance is the completion of the restricted scalar uniqueness basis using an explicit scalar accumulator, without assuming pairing, counter-aware recursion, or equality reflection along the way.

8 Implementation, scope reuse, and validation

8.1 The uniqueness checker

The complete native checker uses a tagged JSON syntax with de Bruijn indices. Terms are `["v",i]`, `["z"]`, `["s",t]`, and `["i",n,a,h]`; only the last child of the iterator binds a numerical variable. Formulas are `["=",s,t]`, `["q",B]`, and `["d",B]`. For D, the closure coordinate is index zero and the witness is index one. A certificate has exactly the fields `variables`, `context`, `goal`, and `proof`. All are checked by the standalone module, including malformed values and Boolean/integer aliases.

The new proof instruction is `["ur",f,h,t,p]`. It validates f and h at one extra numerical slot and t at the original scope. It checks p in the lifted original context, with no added hypothesis, against

$$f(Sz) = h(f(z)).$$

It returns $f(t) = I(t, f(0); h)$. The state slot of h is the sole active slot; the checker does not permit an additional counter slot in the update. Its matched control uses exactly the same syntax, traversals, equality rule, logical rules, JSON interface, and errors, replacing UR by equation-progress introduction. The 71-byte increase is therefore a real cost of this direct rule replacement, not a difference in validation.

The supplied tests exercise correct and incorrect UR recurrences, retention of the original context, absence of an induction hypothesis, freshness of the target, nested iterator binders, passive-parameter shifts, and eigenparameter escape. The 66 process checks run both normal Python and `python -O`; all have their expected outcomes. These are kernel regression checks. They are not a formal verification of the complete metatheorem or certificates for every arithmetic lemma in this report.

8.2 The separate four-byte refinement

The 866-byte program implements the existing scalar calculus, not UR. It composes two existing implementation observations: lexical validation of the entire integer/list certificate, and reuse of the first kernel scope traversal for public-goal bound-variable validation [4, 2]. No numerical axiom or inference rule changes.

The only source change is the following condition in the public grammar validator:

`2 < a < d` becomes `a > 2`.

The remaining checks still establish the public term/formula arities, the permitted leaf roles, and integer/list types. At the initial kernel call, the existing scope map traverses the entire goal before attempting any proof instruction. A nonzero term variable outside its binder depth reaches the map's invalid-substitution sentinel and rejects. Thus an additional goal admitted by the relaxed preliminary test cannot be accepted by the complete checker. Valid goals reach the byte-identical kernel and the same proof stream. This proves equality of the acceptance sets on the documented finite JSON interface; the accompanying detailed argument records the exact sentinel and depth convention.

The improvement is four normalized bytes, two significant tokens, three AST nodes, and three gzip bytes in this implementation. Its relevance is that it is a measured full-program improvement with the same public contract. Its mathematical significance is limited: it is not a new axiomatization, and it does not beat the distinct compact guarded codec. The differential suite makes 332 standalone runs, equally split between ordinary and optimized Python, with 108 acceptances, 224 rejections, and zero mismatches. It includes malformed JSON, unused-prefix type attacks, freshness regressions, and nine goals that pass the relaxed preliminary test but are rejected by the initial kernel scope traversal before any proof packet is consumed. The exact code-change check and the scoped argument about recursion limits are included.

8.3 Arithmetic validation and assurance limits

Two independent standard-natural checks accompany the proof. One checks the complete explicitly defined scalar predecessor construction and then the reflection identities using subtraction implemented by iterating that predecessor. It reports 1884 successful checks. The second checks 477760 instances of the Boolean comparisons, accumulator, telescoping equations, and equality test, with an independent finite-sum oracle for the accumulator and four deliberately incorrect nearby variants that are detected. Bounds and exact counts are recorded in the JSON results.

The universal result rests on the written derivations. Finite tests cannot establish it in nonstandard structures or intuitionistic models. The new equivalence has been reviewed independently at the predecessor, reflection, and logical-bootstrap boundaries, but is not yet formalized in Lean or Rocq. The already existing scalar-to-iPRA bridge is the only proof-assistant verification inherited from the repository. The new checkers are executable artifacts, not proof assistants verifying their own correctness.

9 Further research questions

1. Formalize scalar UR and the explicit accumulator in the existing Lean project, preserving the dependency order: cases and logic, UR predecessor, Boolean diagonal descent, reflection, then general equation induction. The Boolean descent lemma should isolate much of the new proof.

2. Can the complete UR checker beat a matched induction checker after an architectural change that genuinely shares computation and uniqueness? The present direct replacement is larger; that negative measurement should guide the next attempt.

3. Can the uniqueness rule be weakened to a restricted family of updates while retaining the predecessor bootstrap and retraction rigidity? The reflection stage mostly uses constant-recurrence uniqueness, whereas predecessor and retractions use nontrivial updates. A justified reduction would need to cover both uses.

4. Can the explicit scalar accumulator support a more direct primitive-recursion compiler or a shorter predecessor construction? Its recurrence is counter-dependent, yet its definition uses only composition of scalar subtraction and addition. This distinction may help avoid premature tuple coding in other bootstraps.

5. Can a smaller complete presentation be established under one fixed public syntax and validation contract? Current reported minima span different languages and input envelopes. A common contract is necessary before a meaningful lower-bound or absolute-minimality question can be posed.

10 Reproduction and files

The archive includes this standalone `report.tex` and its compiled `report.pdf`; editable proof sections; the two matched uniqueness checkers; the 866-byte existing-calculus checker and its exact control; finite arithmetic and checker tests; measurements and provenance; and the inherited mathematical source files used in the equivalence bridge. No network access is required for reproduction.

Run `python3 reproduce.py` to repeat the supplied measurements and tests. Run `python3 reproduce.py -pdf` to rebuild the report with pdfLaTeX. To compile the standalone source directly, run `pdflatex -interaction=nonstopmode -halt-on-error report.tex` twice. The archive contains no claimed complete Lean project for the new theorem.

References

[1] Vladimir Reshetnikov, *ProveIt: Concise presentations of intuitionistic primitive recursive arithmetic*. Repository snapshot `11be5aabc6b07262046e2553419bc6b05271d178`. https://github.com/VladimirReshetnikov/ProveIt/tree/11be5aabc6b07262046e2553419bc6b05271d178/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] *Concise iPRA: scope reuse and mathematical limits*, incoming package `ipra_scope_kernel_2026-11-11.zip`, especially `sections/uniqueness_note.tex`, in the pinned project snapshot. The exact inherited uniqueness note is reproduced in `reference/`.

[3] *Complementary introductions with one local assumption*, `Research/certified_zero/reference/inherited_choice_basis.tex`, in the pinned project snapshot. Includes the logical reconstruction independent of progress induction.

[4] *Validated presentation*, `Research/validated_presentation/code/native_typed_full.py` and its validator proof, in the pinned project snapshot.

[5] *Zero-normal scalar iteration*, incoming package `ipra_zero_normal_2026-11-11.zip`, in the pinned project snapshot.

[6] *Template-free equality transport*, incoming package `ipra_template_free.zip`, in the pinned project snapshot.

[7] M. Okada and P. J. Scott, *A Note on Rewriting Theory for Uniqueness of Iteration*, Theory and Applications of Categories **6** (1999), no. 4, 47–64. https://tac.mta.ca/tac/volumes/6/n4/6-04abs.html.